

Python Port Scanner - Code Explanation

This document explains a beginner-friendly Python port scanner project used for learning basic cybersecurity and networking concepts.

Python Code

```
import socket
from datetime import datetime

def scan_port(target, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(1)

        result = sock.connect_ex((target, port))

        if result == 0:
            print(f"[OPEN] Port {port}")

        sock.close()

    except socket.gaierror:
        print("Hostname could not be resolved.")
    except socket.error:
        print("Could not connect to server.")

def main():
    print("Simple Python Port Scanner")
    print("-----")

    target = input("Enter target IP address: ")

    start_port = int(input("Enter start port: "))
    end_port = int(input("Enter end port: "))

    print(f"\nScanning {target} from port {start_port} to {end_port}")
    print("Started at:", datetime.now())
    print("-----")

    for port in range(start_port, end_port + 1):
        scan_port(target, port)

    print("-----")
    print("Scan completed.")

if __name__ == "__main__":
    main()
```

1. Import Modules

The socket module is used to create network connections. The datetime module is used to display the scan start time.

2. scan_port() Function

This function checks whether a specific TCP port is open on the target system. It creates a socket connection and attempts to connect to the selected port.

3. socket.socket()

AF_INET means IPv4 addressing. SOCK_STREAM means TCP communication. TCP is commonly used for services such as websites and SSH.

4. connect_ex()

connect_ex() attempts to connect to the target port. If the return value is 0, the port is open. Otherwise, the port is closed.

5. Error Handling

The scanner handles invalid hostnames and network errors using try-except blocks.

6. Main Function

The main() function asks the user for the target IP address and port range, then scans each port one by one.

7. Port Scanning Logic

The program loops through the selected port range using a for-loop and calls scan_port() for each port.

8. Cybersecurity Concepts Learned

This project helps understand TCP networking, sockets, open and closed ports, basic reconnaissance concepts, and Python networking programming.

Important Ethical Note

Only scan systems that you own or have permission to test. Unauthorized scanning may violate laws or organizational policies.

Example Output

```
Simple Python Port Scanner
-----
Enter target IP address: 127.0.0.1
Enter start port: 20
Enter end port: 100

[OPEN] Port 22
[OPEN] Port 80

Scan completed.
```